Time series

Chicago transit ridership

LECTURE 15 | GÉRON CH. 13

Applications of Machine Learning

BSc Mathematics of Artificial Intelligence · Third year

Where we are

Last lecture asked *what is where, and how many*:

- A detector returns boxes, scores and labels; IoU scores the overlap of two boxes, and is exactly zero, with zero gradient, once they stop touching — GIoU puts the empty space back
- mAP is a mean of a mean: average precision per class, then averaged over IoU thresholds
- The count the stakeholder asked for moves with a score cut-off nobody chose and with non-maximum suppression, another knob
- Segmentation swaps the box for a mask, scored by the same argument

NOT HERE

Every lecture so far assumed the rows were exchangeable, so a random split was valid. In a time series the rows are **ordered**, and neighbouring rows are alike: split at random and a point's own near-future lands in the training set — so the score you report is optimistic, by a margin we will measure.

Today

1. The brief, the series, and what a plot of it already tells you
2. **The mathematics** — weak stationarity, differencing, and why copying last week should be hard to beat
3. **The method** — the metric, the baseline to beat, and a linear model on 56 lags
4. **Honest evaluation** — a random split against forecasting forward, and backtesting on a rolling origin
5. **Design exercise** — a hospital-software company's staffing-forecast feature for emergency-room rotas, in nine steps, with the pitfalls and how to address them

Open Lecture 15 in Colab

You have just been hired

You are a data scientist at the transit authority of a large city.

> **THE TASK**
>
> Forecast the number of people who will board **rail** and **bus** on the following day. You have daily boarding totals going back twenty years.

One number per mode, per day, one day ahead.

The first question is never technical

What decision does this number change?

- How many train sets leave the depot
- How many drivers are rostered, and at what notice
- Whether a spare crew is held at each terminus

Those decisions are made the evening before. That fixes the horizon at **one day**, and it fixes when the forecast has to exist: *before* the day it describes.

The two errors are not equal

Forecast too high

Empty vehicles run. The cost is fuel, crew hours and a line in the budget. It is measurable and it is survivable.

Forecast too low

Passengers are left on platforms at peak. The cost is political, and it does not appear in any table you will be shown.

✗ We are about to choose a symmetric metric anyway, and we should say so out loud rather than discover it later.

Frame the problem

Question	Answer	Because
Supervision	✓ **supervised**	the day arrives, and then we know
Task	✓ **regression**	a count, not a category
Learning mode	✓ **batch, offline**	one row per day; the whole record fits in memory

> ***THE LABEL IS FREE***
>
> *Nobody has to annotate anything. The next day's value is the label for today's window. Chapter 13 calls this the trick that makes a series into a supervised dataset.*

Fetch it with a function, not a download

```
from pathlib import Path
import tarfile, urllib.request
import pandas as pd

def load_ridership():
    tarball = Path("datasets/ridership.tgz")
    if not tarball.is_file():
        Path("datasets").mkdir(parents=True, exist_ok=True)
        url = "https://github.com/ageron/data/raw/main/ridership.tgz"
        urllib.request.urlretrieve(url, tarball)
        with tarfile.open(tarball) as t:
            t.extractall(path="datasets", filter="data")
    return pd.read_csv(
        "datasets/ridership/CTA_-_Ridership_-_Daily_Boarding_Totals.csv",
        parse_dates=["service_date"])
```

Same shape as the loader in the first application. The data will change; you will be on another machine; write the function.

Four lines of tidying, and one of them matters

```
df = load_ridership()
df.columns = ["date", "day_type", "bus", "rail", "total"]
df = df.sort_values("date").set_index("date")
df = df.drop("total", axis=1)      # it is just bus + rail
df = df.drop_duplicates()          # and this one removes 62 rows
```

The file ships with **7,701** rows and **7,639** distinct ones. Sixty-two days appear twice, identically. Nothing warns you.

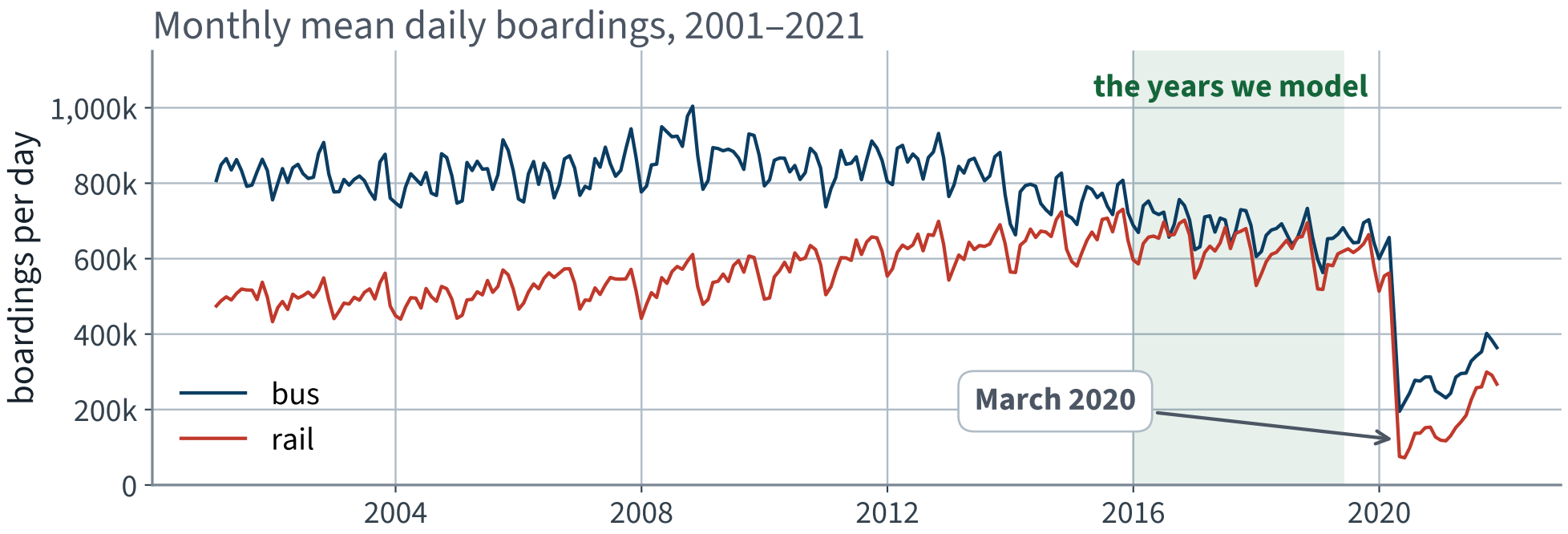
✓ What was dropped — rows, columns, NaNs? Count them. **Here the answer is 62, and you only know because you counted.**

The one categorical column

`day_type`	Meaning	Days
`W`	an ordinary working day	5,336
`U`	the second weekend day, and public holidays	1,216
`A`	the first weekend day	1,087

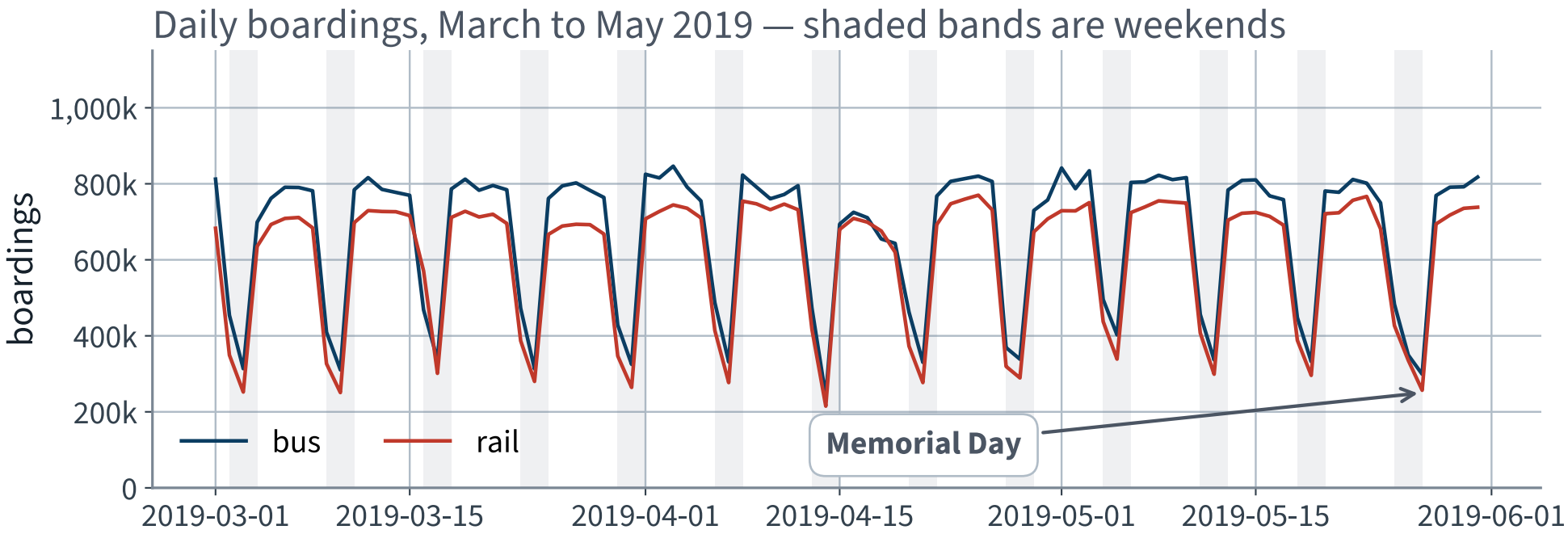
This column is unusual and worth pausing on: we know it **in advance**. The next day's type is on the calendar today, so it is a legitimate input to a forecast of that day. Most features are not like that.

Plot it before you do anything else



Bus falls, rail rises and then falls, and March 2020 is a cliff, not a dip. We model one regime: 2016 to the end of May 2019, 1,247 days.

Now zoom in until you can see individual days



A seven-day cycle so strong that it dominates everything else on the plot.

Observation 1 · A seven-day cycle

- Working days sit near the top of the range
- The two weekend days drop to roughly half
- The pattern repeats **every seven days**, almost exactly

> A repeating pattern with a fixed period is called **seasonality**. The period here is a week, not a year — the word has nothing to do with the seasons.

✓ This immediately suggests a forecast that requires no model at all. Hold that thought.

Observation 2 · One trough is in the wrong place

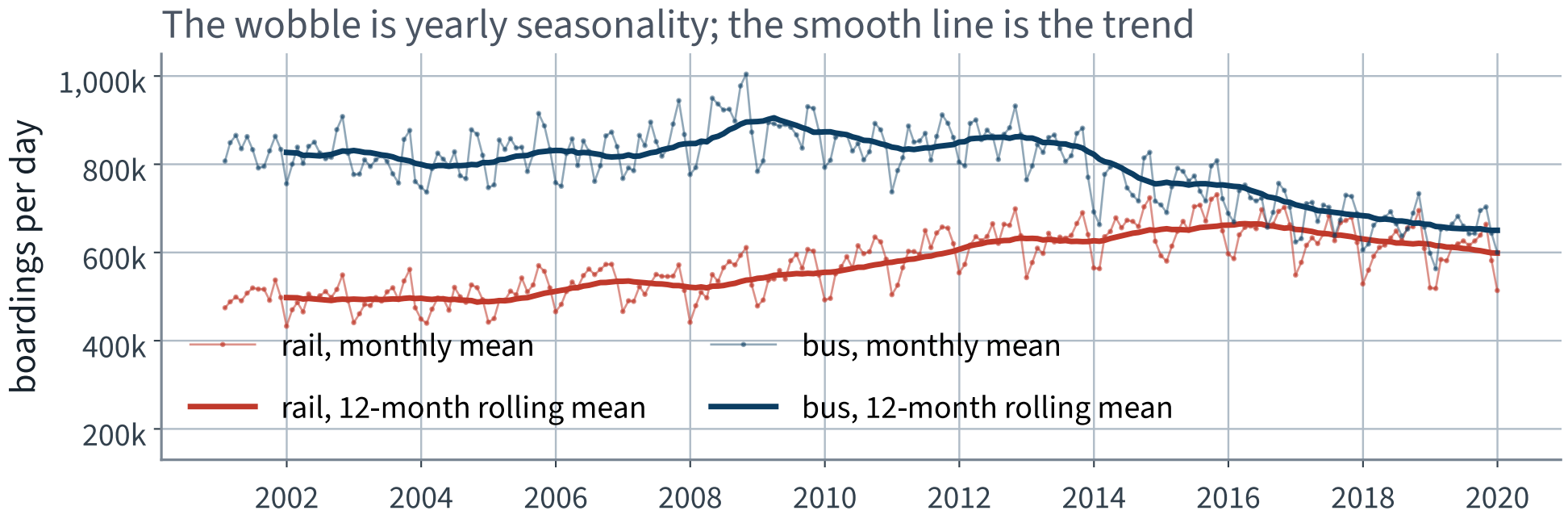
At the end of May there is a deep trough in the middle of a working week. Ask the data rather than the calendar:

```
>>> list(df.loc["2019-05-25":"2019-05-27"]["day_type"])
['A', 'U', 'U']
```

Three consecutive non-working days: a public holiday extending the weekend. The column we were given already knows.

Every anomaly you find should end either in an explanation or in a note that you could not find one. Not in silence.

And a second, slower cycle



The thin line wobbles once a year; the thick line is where the level is actually going.

Observation 3 · Seasonality and trend are different things

Seasonality

A pattern that repeats with a fixed period. Here: seven days, strongly; twelve months, weakly.

Trend

A slow movement of the level itself, with no period. Here: rail rising to about 2015, then falling.

A moving average of the right width removes the seasonality and leaves the trend. That is the whole content of the thick line.

The mathematics takes both of them out arithmetically, and that turns out to explain something about today.

Weak stationarity

A model assumes the rows it will meet are the same kind of thing as the rows it was trained on. For a series $\{y_t\}$ that assumption is **weak stationarity**, three conditions:

$$\mathbb{E}[y_t] = \mu \quad \text{(constant)}$$

$$\mathrm{Var}(y_t) = \sigma^2 < \infty \quad \text{(constant)}$$

$$\mathrm{Cov}(y_t, y_{t+k}) = \gamma_k \quad \text{(depends on } k \text{ only)}$$

Only the first two moments, and only the requirement that the covariance forgets *when* and remembers only *how far*

Differencing

Define the difference operator, and its seasonal version at lag s:

$$(\nabla y)_t = y_t - y_{t-1} \qquad (\nabla_s y)_t = y_t - y_{t-s}$$

Applied to a sampled function, ∇ is a discrete derivative: it returns the slope at each step, so a linear trend becomes a constant. ∇_7 compares each day with the same weekday a week earlier, so a seven-day cycle cancels.

Values are lost at the start each time you apply it. Count those losses; the book's `diff` returns `NaN` there and says nothing.

Why copying last week should be hard to beat

Write $\rho_k = \mathrm{Cov}(y_t, y_{t-k})/\sigma^2$, the **autocorrelation** at lag k. Under weak stationarity,

$$\mathrm{Var}(y_t - y_{t-k}) \;=\; \sigma^2 + \sigma^2 - 2\sigma^2\rho_k \;=\; 2\sigma^2\,(1 - \rho_k)$$

So differencing at lag k lowers the variance only if $\rho_k > \frac{1}{2}$. On our series $\rho_1 = 0.419$, so the first difference makes it *more* variable. The forecast "copy the value from seven days ago" has error $y_{t-7} - y_t = -(\nabla_7 y)_t$: its error *is* the seasonal difference, with variance $2\sigma^2(1 - \rho_7)$. On our series $\rho_7 = 0.831$, so that is $0.34\,\sigma^2$.

> It should be hard to beat for one reason only: ρ_7 **is large**. Not a lucky heuristic — the autocorrelation function, read off at a single lag.

State the prediction precisely first

$$\hat{y}_{t+1} = f(y_{t-55}, y_{t-54}, \ldots, y_t)$$

Given the last eight weeks of daily boardings up to and including today, predict the next day. One number out.

Eight weeks is a choice, not a law: a whole number of weeks, so every position in the window sees the same weekday each time.

Three metrics, and what each one buys

$$\mathrm{MAE} = \frac{1}{m}\sum_{i}\left|\hat{y}_i - y_i\right| \qquad \mathrm{RMSE} = \sqrt{\frac{1}{m}\sum_{i}\left(\hat{y}_i - y_i\right)^2}$$

$$\mathrm{MAPE} = \frac{1}{m}\sum_{i}\frac{\left|\hat{y}_i - y_i\right|}{y_i}$$

RMSE punishes a large miss quadratically; MAE does not; MAPE is relative and therefore unit-free.

✓ We choose MAE: it is in passengers, and a handful of holidays cannot dominate it. MAPE goes beside it, because a percentage is what a stakeholder repeats back to you.

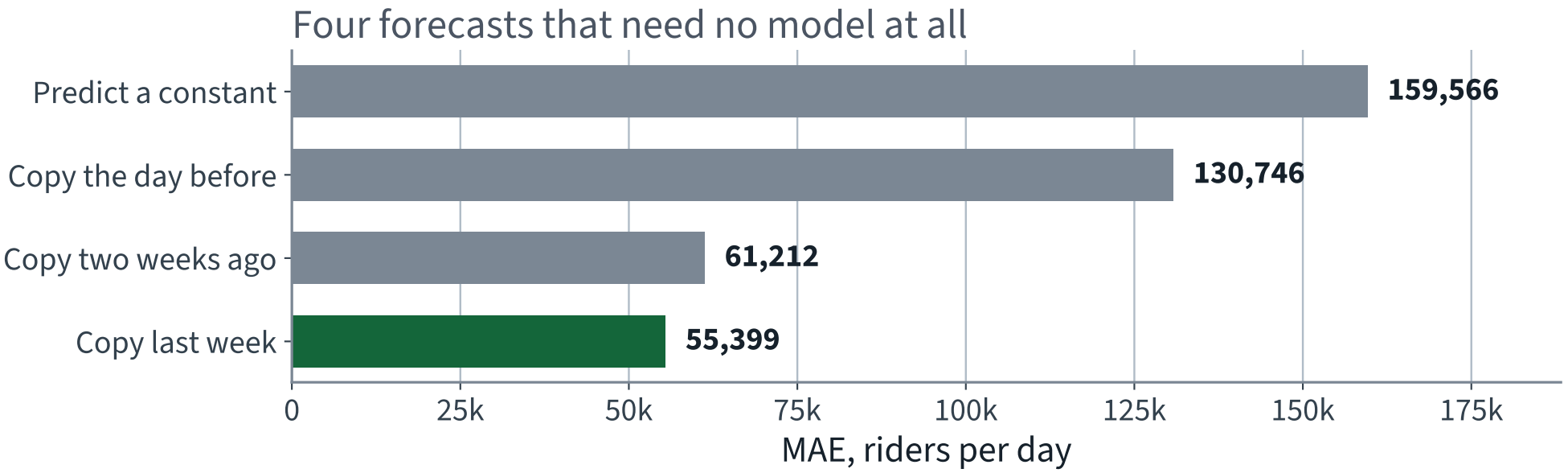
One trap in MAPE, stated before we use it

MAPE divides by the truth. On a day when ridership collapses, the denominator collapses too, and a modest absolute miss becomes an enormous percentage.

> **WHERE THIS BITES**
>
> Exactly on the holidays, which is exactly where the model is worst. A metric that is most sensitive where the model is weakest will make every improvement look like noise.

We use it as a second opinion, never as the objective.

What nothing scores



Copying last week is **2.4 times** better than copying the previous day. The seven-day cycle is worth more than recency.

The number to beat

55,399

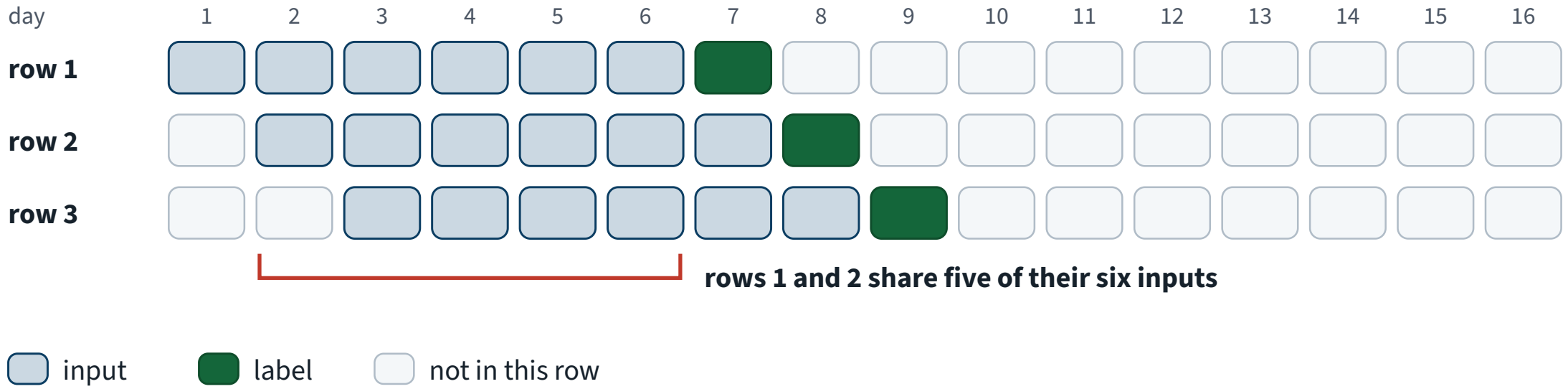
Mean absolute error of *copy the value from seven days ago*, over 1,191 days.

That is **12.0%** of a typical day. On the three months the book plots it is better still: **42,143** for rail (9.0%) and **43,916** for bus (8.3%).

One series becomes a table of overlapping rows

Turning one series into a table of rows

Each row is a window of past days; its label is the day that follows.



drawn with a 6-day window; the lecture uses 56, so consecutive rows share 55 of 56

Inside a row, nothing looks forward. Whether that still holds across rows is decided by the split, not by this table.

Every row is the row above it, shifted by one day. They overlap almost completely.

The dataset class

```
import torch

class TimeSeriesDataset(torch.utils.data.Dataset):
    def __init__(self, series, window_length):
        self.series, self.window_length = series, window_length

    def __len__(self):
        return len(self.series) - self.window_length

    def __getitem__(self, idx):
        end = idx + self.window_length     # first index after the window
        return self.series[idx:end], self.series[end]
```

Nine lines. The subtlety is entirely in `__len__` and in the single index `end`, which is both the end of the window and the position of the label.

The simplest thing that could work

$$\hat{y}_{t+1} = b + \sum_{k=0}^{55} w_k \, y_{t-k}$$

Fifty-six weights and a bias. If the model learns $w_6 = 1$ and every other weight zero, it has reinvented *copy last week*.

✓ That is a useful thing to know before you start: the linear model contains the baseline. It cannot do worse unless we fit it badly.

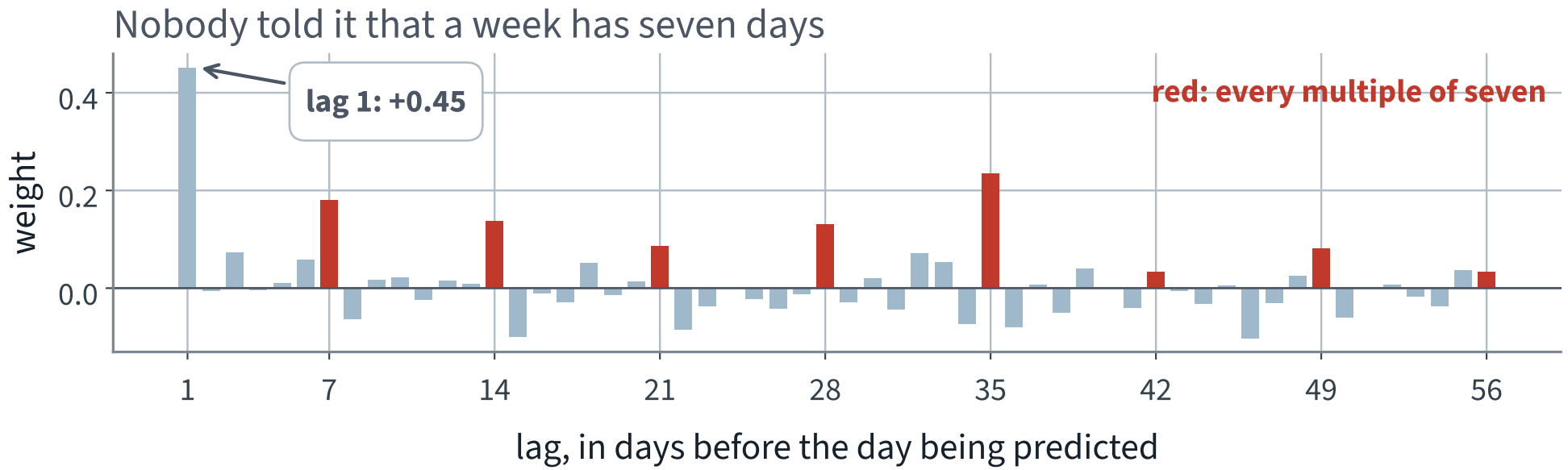
The linear model, measured

44,925

Against **55,399** for copying last week, under shuffled five-fold cross-validation.

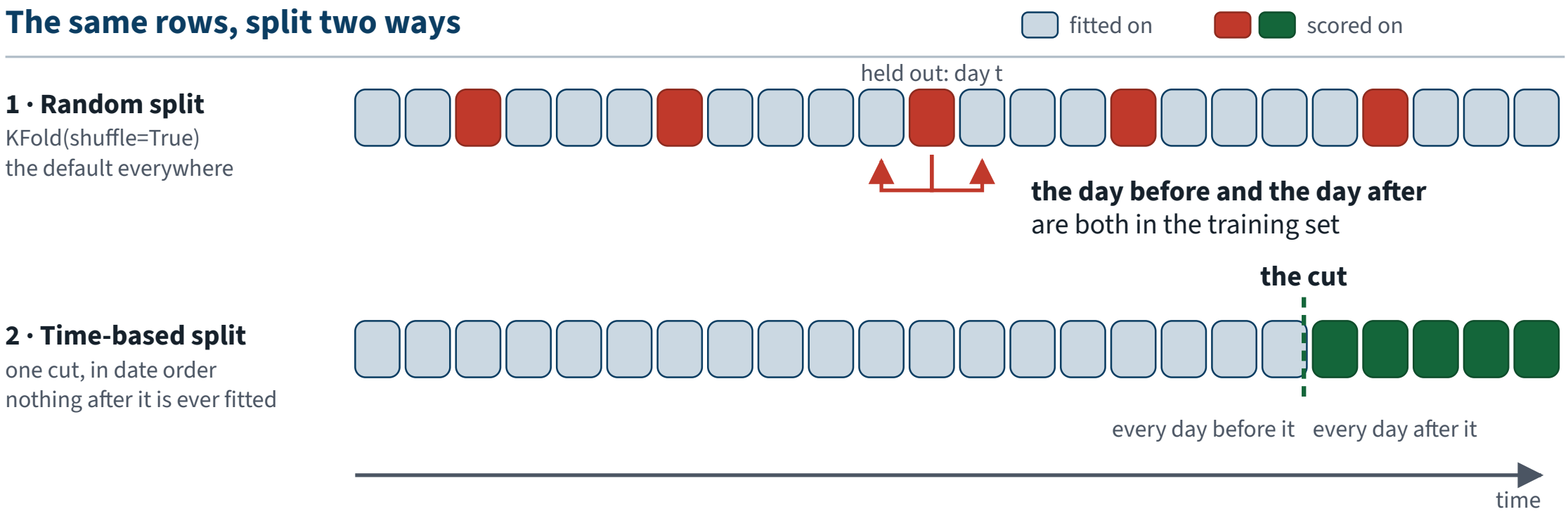
18.9% better, and the five folds run from 39,538 to 51,096. Over twenty different shuffles the mean moves by about 300, so the improvement is far larger than the noise in the estimate.

Fifty-six weights, and where they went



Every red bar is a multiple of seven, and every red bar is positive. That is the weekly cycle, learned rather than supplied.

That score came from a shuffled split



✗ The upper protocol asks the model to fill a gap. The lower one asks it to forecast. Only one of those is the job.

The argument, stated properly

Cross-validation estimates the risk

$$R(f) \;=\; \mathbb{E}_{(X,Y)}\left[\ell(f_{\mathcal{D}}(X),\, Y)\right], \qquad (X,Y) \perp \mathcal{D}$$

It is unbiased for R under two conditions:

1. the held-out rows are **independent** of the training rows
2. the held-out rows are drawn from the distribution the model will meet **in deployment**

✗ A shuffled split of a time series satisfies neither.

Condition 1 fails, and ρ_k says by how much

$\mathrm{Cov}(y_t, y_{t+k}) = \sigma^2 \rho_k$, which is $0.83\,\sigma^2$ at $k = 7$ and never near zero for $|k| \leq 56$. The rows are dependent by construction.

> Conditioning on neighbours reduces the variance of what remains to be predicted. With $\rho_k > 0$, a model that has been fitted on rows *surrounding* a held-out row faces a smaller conditional variance than one fitted only on rows before it.

Smaller conditional variance means smaller achievable loss, so the estimate is biased **downwards**. Positive autocorrelation fixes the direction of the bias: optimistic in the overwhelming majority of cases.

✓ Not "always". That would be a theorem, and we have not proved one — the argument above is a heuristic about conditional variance, not a bound. What we have is a measurement: on this series, 9.0% after matching training size, test size and the baseline.

> Which is precisely why the slides that follow control for those alternatives rather than asserting the direction. A lecture whose subject is not asserting things should not assert this one either.

Condition 2 fails, and it is not subtle

In deployment, *every* training day precedes *every* day being forecast.

Under a shuffled split, on average half the training days come **after** the day being scored.

> **THE NAME OF THIS LECTURE**
>
> The model is not forecasting. It is being handed the shape of the series around the point it is asked about, and interpolating. That is a different and much easier problem, and nobody is paying for it.

The split the system will actually face

Fit on everything up to the end of 2018; forecast the five months that follow, one day at a time, never seeing any of them.

Model	Random split	Forecasting forward	Gap
Linear, 56 lags	44,925	✗ 56,446	+25.6%

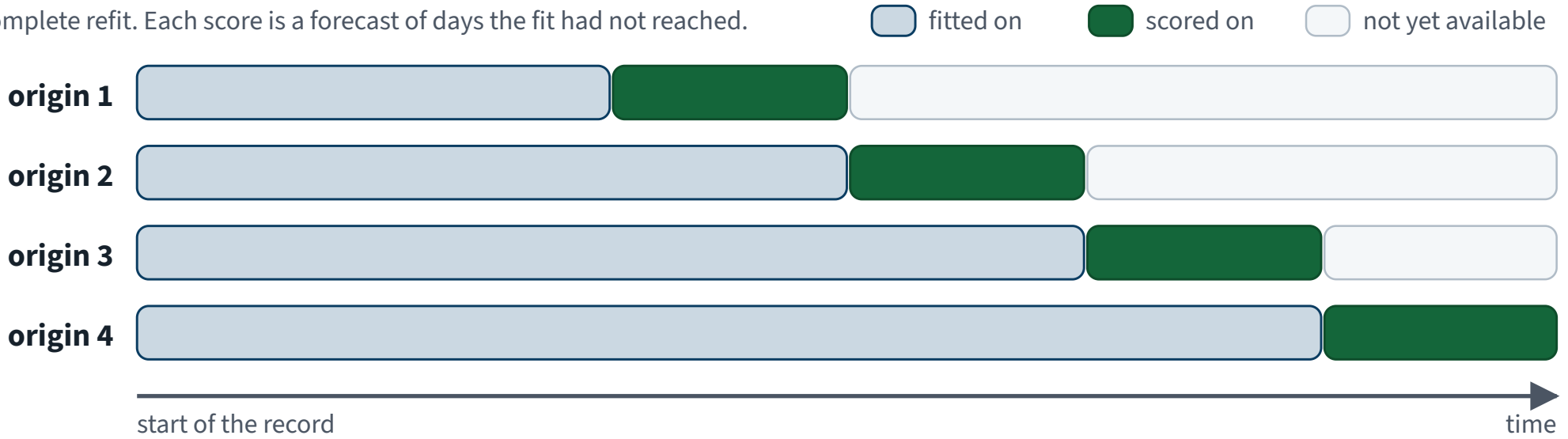
+25.6%

The same model's error, once it has to forecast forward. Not all of it is the split — the next slides take it apart.

Rolling-origin backtesting

Backtesting: re-run the build at four origins

Each row is a complete refit. Each score is a forecast of days the fit had not reached.

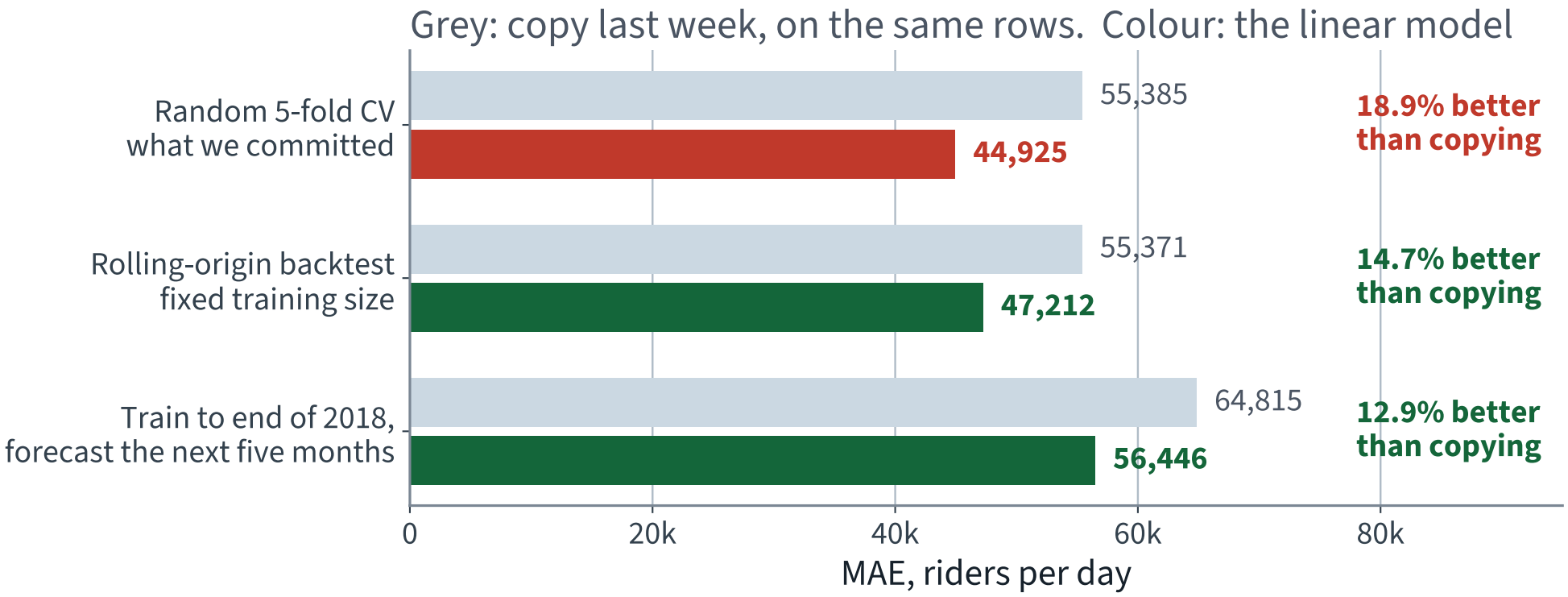


What you get
Four out-of-sample scores instead of one, so you can see the spread as well as the level.

What to watch
Origin 1 is fitted on a quarter of the data. Its score is not comparable.

Four refits, four out-of-sample scores, and a spread you could not get from a single cut.

Put the baseline beside every protocol



Copying last week needs no fitting, so it measures only how hard the scored rows are. Against it, the margin falls from **18.9%** to **12.9%** as the protocol becomes honest.

The controlled comparison

Changing the protocol also changes how much the model trains on. Hold it fixed: same training size (700 rows), same test size (98 rows), same model — only the *arrangement* differs. Score each against copying last week **on its own rows**, which cancels the difficulty of the period:

Protocol	Model	Naive	Ratio
random draws of 98 rows, 20 of them	48,093	61,461	**✗ 0.782**
rolling origin, 5 origins	47,212	55,371	**0.853 ✓**

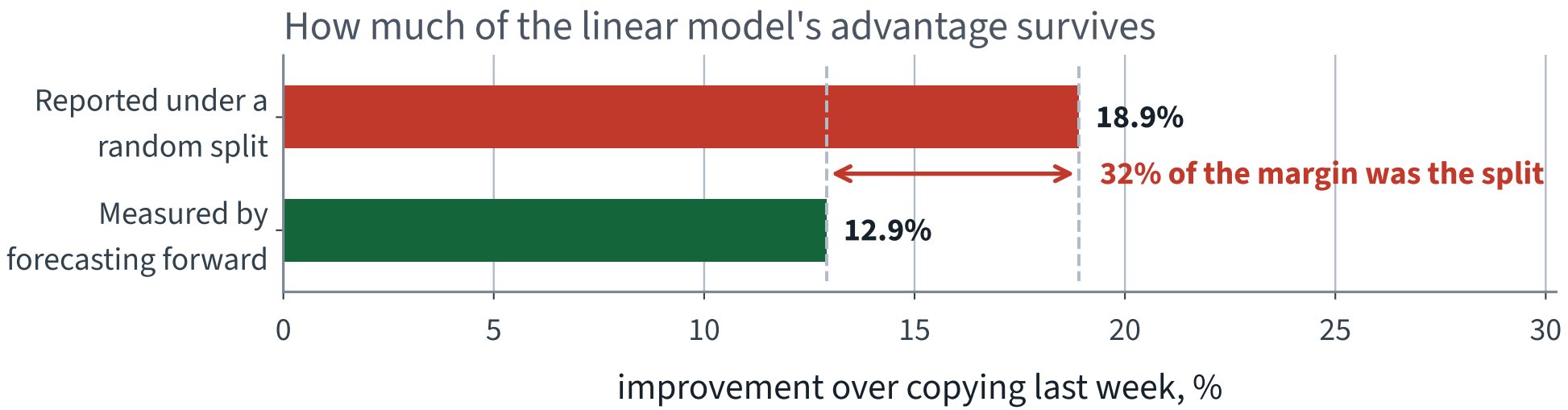
The random split reports the model as **9.0%** more skilful than it is.

The honest verdict, in three numbers

- **25.6%** — the raw gap on the linear model's number. This is what you would have been wrong by, and it is real
- **17%** of that is the forecast period genuinely being harder; the baseline moves too
- **7.4%** is the protocol itself: how much the random split overstates the linear model's skill, with each protocol's baseline computed on its own rows. Match the training size and the test size as well and it is **9.0%**, so it survives every control we could apply

The recurrent network the next lecture builds will be measured the same way — forward in time, against copy-last-week on the identical rows — so its number can be read against this one.

Put it in the terms the stakeholder will hear



✗ We told them our linear model was 18.9% better than doing nothing. It is 12.9% better. A third of the case for building it was an artefact of the evaluation.

One refinement, named but not used

Even a clean cut leaks a little: the last training window overlaps the first scored day's history. The repair is to leave a gap — **purging** the overlapping rows and adding an **embargo** of a window length after the cut.

> On this data the effect is small, because the overlap is 56 rows out of 1,040. On a dataset where windows are long relative to the record, it is not.

Purging and embargo are standard practice in financial backtesting and are **outside the book**; not examinable. The idea — that adjacency itself is a channel — is examinable, because we derived it.

The notebook for this lecture

Open Lecture 15 in Colab

- **Runs on free CPU**
- The data tidied and counted, the series plotted, and the pool fixed before any model
- The autocorrelation at lags 1, 7 and 14, and the variance of each difference predicted from it before it is measured
- The baselines, a linear model on 56 lags, and that model scored under a shuffled split, forecasting forward, and on a rolling origin

What to change: shuffle the rows before splitting and re-run the evaluation — the failure condition below, produced by you rather than quoted at you.

> **FAILURE CONDITION — A SHUFFLED SPLIT ON ORDERED ROWS**
>
> Shuffle the rows before splitting and the score improves, because every test point is surrounded in time by its own neighbours. Nothing warns. What is reported as a forecast is a measurement of interpolation.

Design exercise · in the manner of an ML system-design interview

The task: a hospital-software company’s staffing-forecast feature

> **THE BRIEF**
>
> A hospital-software company sells hospitals a rostering system; a new feature forecasts **emergency-room arrivals per shift** from the customer’s own record, so the planner staffs each shift in advance. The business measures renewals, the agency cover bought when a forecast ran low, and the staffing rules it now answers to.

- **Given:** years of timestamped arrivals, so a count per shift, three a day; the holiday calendar; a roster fixed a fortnight ahead
- **Scale:** a few hundred arrivals a day per department; hundreds of customers; a new week of truth every week
- **Consumers of the output:** the planner, the bed manager, the agency that covers at short notice

✓ Nine steps follow, each with the pitfall that catches a first attempt and what to do about it. The model at the centre is today’s: a linear model on lagged counts, judged against copying last week, forward in time.

Step 1 — Frame it: the horizon the planner needs, and the metric in arrivals

- The decision is the roster, drawn a fortnight before the first shift it covers: the forecast for every shift of it must exist on the day the roster is drawn
- Supervised regression, one row per shift; the label is free — the shift happens, and then we know
- MAE in arrivals per shift, with MAPE beside it for the planner; the asymmetry said out loud — an under-forecast leaves patients in the corridor, an over-forecast pays an idle nurse — because the metric is symmetric

> **PITFALL — THE WRONG HORIZON**
>
> The first prototype forecasts the next shift from the one just finished, and the demo impresses; the planner cannot use it, because the roster was fixed a fortnight earlier. Every shift of the horizon must be forecast from what was known on the roster's day and scored at that distance: the error a fortnight out is the number, not the error one shift out.

Step 2 — The series: plot it, and name what repeats

- Years of counts per shift, plotted before anything else: a cycle across the day's three shifts, a weekly one, a yearly one with the winter peak, a level drifting as the catchment grows
- Weak stationarity — constant mean and variance, a covariance depending on the lag only — is what a model assumes; the drift breaks it. A difference turns the drift into a constant, a seasonal difference at one week cancels the weekly cycle
- The sample autocorrelation at a lag of one week says, before any model, how hard copying last week will be to beat

PITFALL — A TREND TREATED AS NOISE

A model fitted on a level that is rising is biased on the future, not noisy: the error accumulates, every roster there is short by the same growing margin, and the agency invoices say so. Check the mean by year before fitting, difference or fit on the current regime, and keep the drift on the chart the planner sees.

Step 3 — Split forward in time, and backtest

- Fit on everything before a cut; forecast the shifts after it, at the planner’s horizon, never seeing them. Then move the cut forward several times — a rolling origin — for several scores and their spread
- The seasonal naive beside every protocol, on the same rows: it needs no fitting, so it measures only how hard the scored weeks were

PITFALL — A SHUFFLED SPLIT ON ORDERED ROWS

Neighbouring shifts are almost the same measurement made twice. Shuffle, and each held-out shift is surrounded in training by its own neighbours: the model interpolates, the score is optimistic by a margin, and the margin is largest exactly where the autocorrelation is. Part of the sales case is then an artefact of the split. Forward cut, rolling origin, the naive alongside.

Step 4 — Features: only what is known on the roster's day

- A window of past counts ending on the day the roster is drawn, a whole number of weeks long
- The calendar — day type, public holiday, school term — is known in advance, so it is a legitimate input for every shift of the horizon; most features are not
- Weather: the forecast available on the roster's day, if any; never the weather that happened

PITFALL — LEAKAGE OF THE FUTURE THROUGH A FEATURE

"Arrivals the previous shift" is a fine feature one shift ahead and does not exist for a shift a fortnight away; the week's mean, the ambulances that day, the bed occupancy that morning — each is a number from after the roster was drawn, however near to hand. Build every feature as of the roster's day, with the code that builds it in service, and lag each by at least the horizon.

Step 5 — The model: the baselines first, then a linear model that contains them

- Predict the mean; copy the same shift the day before, or a week before. None is machine learning, and the product could ship any of them this afternoon
- A linear model on the window of lagged counts plus the calendar, one fitted for each distance in the fortnight with the label moved that far ahead: if it learns a weight of one on the lag of a week and nothing else, it has reinvented the seasonal naive, so it cannot do worse unless it is fitted badly

PITFALL — THE BASELINE MISSING FROM THE TABLE

The linear model's error on its own looks like a result. Against copying last week on the same rows it is a margin, and sometimes none: the seasonal naive is strong because the week's autocorrelation is high, and a customer whose series the model does not beat it on is shown the naive, and told so. Report the ratio to the seasonal naive, per horizon, on identical rows.

Step 6 — Failure conditions: holidays, and the day a hospital changed

- Holidays: surgeries close, arrivals jump, and the calendar knew. The day type is an input, and the holiday error is reported apart, because MAPE's denominator and RMSE's square let a handful of days dominate
- Regime changes: a neighbouring department closes, a pandemic, a new triage rule — the level moves and stays moved. Plot the backtest error through time: a persistent step is a regime, not a bad week

PITFALL — FITTING ACROSS THE CLIFF

A model fitted over a record that contains a regime change describes two hospitals and is biased on both, the backtest averages it away, and nobody opens that one customer's chart. Fit on the current regime only, keep the earlier record for the seasonal shape, and let a persistent step in the forward error raise an alert.

Step 7 — Evaluation: an interval the planner can staff to, not a point

- The planner does not staff to the mean: the question is how many nurses cover the shift on all but a few shifts in a hundred
- The rolling backtest leaves a set of errors at each horizon: the forecast plus their quantiles is an interval, and it widens with the distance
- Report coverage — the share of held-out shifts that fell inside the interval — forward in time

> **PITFALL — A POINT FORECAST HANDED OVER AS A PLAN**
>
> The planner rounds the mean up by habit and either over-staffs every shift or is short on the busy ones; the product is blamed for both, and the staffing record shows it. Hand over a lower and upper bound from the backtest errors at that horizon, and check their coverage on the forward split before a roster uses them.

Step 8 — In service: retraining as the series moves

- Each week brings a new week of truth: refit on the record up to now, and score the published roster against what arrived — the backtest, live
- A few customers first; the seasonal naive alongside every week: when the margin over it shrinks, the data team on call is paged — stale model or moved series

PITFALL — A MODEL FITTED ONCE AND LEFT

The catchment grows, a surgery closes, winter arrives early; a fixed model's error turns from noise into bias, and the rosters are short by a growing margin that nobody plotted. Refit on a schedule, chart the forward error per horizon, and treat a persistent step in it as the regime change it is.

Step 9 — Ship it: the pipeline, and the pitfalls in one table

Arrivals log → count per shift → plotted, cycles and drift named → forward cut, rolling origin → lags and calendar as of the roster's day → seasonal naive, then a linear model per horizon → holidays scored apart, the regime chosen → intervals from the backtest errors, coverage checked → refit weekly, the naive as alert.

Pitfall	Address it by
The wrong horizon	forecasts from the roster's day, scored at that distance
A drifting level treated as noise	the mean by year checked; difference, or fit on the regime
A shuffled split	forward cut, rolling origin, the naive on the same rows
A feature from after the roster's day	features as of that day, lagged by at least the horizon
No baseline in the table	the ratio to the seasonal naive, per horizon
Fitting across a regime change	the current regime only; a persistent error step as alert
A point forecast as a plan	an interval from the backtest errors, with its coverage
A model fitted once	refit on a schedule, the forward error charted

What we did

1. Saw that ordered, autocorrelated rows break the assumption every previous lecture rested on
2. Defined weak stationarity, and differencing: once for a trend, at lag seven for the weekly cycle
3. Derived $\mathrm{Var}(y_t - y_{t-k}) = 2\sigma^2(1 - \rho_k)$, and used it to predict — before measuring — that copying last week would be hard to beat
4. Chose MAE, measured the baselines, and fitted a linear model on 56 lags that found the week on its own
5. Measured the optimism of a random split on autocorrelated data, and replaced it with forecasting forward and backtesting on a rolling origin
6. Designed the staffing-forecast feature a hospital-software company sells with its rostering system in nine steps — the horizon set by the roster, the drift and the cycles named before any model, a forward split on a rolling origin, features as of the roster's day, the seasonal naive on identical rows, the regime chosen and holidays scored apart,

NOT PRESENTED — FOR AFTERWARDS

Exercises Lecture 15

five, in the style of the written paper

Exercises — Lecture 15

1 For a stationary series, $\mathrm{Var}(X_t - X_{t-h}) = 2\gamma(0)(1 - \rho(h))$. State the condition on $\rho(h)$ under which differencing *increases* the variance. [4]

2 Explain why the forecast "copy the value from seven days ago" is hard to beat on daily ridership, in terms of the autocorrelation function. [4]

3 State why MAPE is the wrong metric for transit ridership, using a specific day as the example. [4]

Solutions on Lecture 16's deck.

Exercises — Lecture 15 (continued)

4 A cross-validated forecast uses `KFold(shuffle=True)` and reports a good score. Name the two conditions the split violates. [5]

5 A dataset class turns a pool of 1,247 daily values into windows of 56 days, each labelled with the day that follows it. How many rows does it yield, and what goes wrong if the label is taken one index earlier? [4]

Solutions on Lecture 16’s deck.

THE FIVE SET AT THE END OF LECTURE 14

Solutions Lecture 14

not part of this lecture

Solutions — Lecture 14

1. Two boxes are disjoint and moved further apart. State what IoU and its derivative do, and why no learning…

✓ Both are identically zero; a constant has no descent direction.

- Past contact the intersection is exactly zero for every separation
- The gradient is absent rather than small, so no optimiser can act on it

2. A detector counts objects with MAE 3.00 at the conventional score cut-off of 0.5. A sweep over the same 128…

✓ The cut-off was chosen on the same images the error is reported on, so 1.92 is optimistic by construction; quote it as a diagnostic, beside the 3.00 at the threshold fixed before looking.

- Picking 0.75 after seeing the error curve is fitting a hyperparameter on the test data
- With no held-out images, only a threshold fixed in advance gives an honest figure

Solutions — Lecture 14 (2)

3. Average precision is defined with a maximum inside it. State what that maximum repairs, and which earlier…

✓ It replaces the non-monotone precision by its running maximum — the non-monotonicity proved in Lecture 3.

- Precision falls at every false positive and can rise again
- Integrating the raw curve would reward the sawtooth rather than the ranking

4. mAP is a mean over classes of a mean over IoU thresholds. Give one thing each averaging hides.

✓ The class mean hides that a one-instance class weighs as much as a 350-instance one; the threshold mean hides the spread between loose and tight matching.

- An unweighted mean ignores how many instances each class has
- A single number in the middle stands for two very different regimes

Solutions — Lecture 14 (3)

5. Your mAP over IoU 0.50 to 0.95 on 128 COCO images is 43.9; torchvision reports 37.0 for the same weights on…

✓ Run a reference evaluator on the same predictions and annotations (ours agreed to 0.0014); the gap is then the sample — 128 images, many easy, and classes with a single instance.

- An implementation you wrote yourself is checked against one you did not before its number is quoted
- A small evaluation set moves the score, here in the flattering direction, with the detector unchanged

LECTURE 16 · GÉRON CH. 13

Recurrent networks

A recurrent cell, unrolled, and trained by backpropagation through time
Forecasting several steps ahead, and a ranking that flips between runs
LSTM and GRU gates, the final table against copying last week, and the 2020 regime change

Run the Lecture 15 notebook first